

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	G.RAJU	Reg. No.:	192411065
Section / Batch:	B. TECH(CSE)	Date:	12-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Case Study: E-Commerce Product Sorting System (Merge Sort)

Problem Statement

An e-commerce platform stores millions of products with different prices and ratings. To provide fast search results and accurate product listings, the products need to be sorted efficiently.

Analysis

The system must handle very large datasets while maintaining the correct order of products with the same price or rating. Since products are frequently updated, the sorting algorithm should be scalable and provide consistent performance.

Key Issues

- Sorting millions of products efficiently.
- Maintaining the relative order of products with equal prices (stability).
- Scalability for increasing data volume.
- Consistent performance regardless of input order.

Divide and Conquer Approach

Merge Sort divides the product list into two equal halves recursively until each sublist contains one element. The sorted sublists are then merged step by step to produce the final sorted product list.

Time Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n \log n)$
- **Space Complexity:** $O(n)$

Proposed Solution

Implement Merge Sort to sort products based on price and rating. The algorithm guarantees stable sorting, ensuring that products with the same price remain in their original order.

Justification

Merge Sort is suitable because it provides consistent performance, supports stable sorting, and efficiently handles very large datasets, making it ideal for e-commerce platforms.

Q2. Case Study: Real-Time Ranking System (Quick Sort)

Problem Statement

A real-time ranking system continuously updates user scores and displays a leaderboard that must remain sorted with minimal delay.

Analysis

The system requires a fast sorting algorithm capable of handling frequent score updates. Poor pivot selection can significantly reduce performance.

Key Issues

- Frequent updates to rankings.
- Efficient pivot selection.
- Avoiding worst-case performance.
- Fast response time.

Divide and Conquer Approach

Quick Sort selects a pivot element, partitions the data into smaller and larger elements, and recursively sorts both partitions until the list is completely sorted.

Time Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n^2)$

Proposed Solution

Use Quick Sort with a randomized pivot selection strategy. Random pivot selection minimizes the chance of highly unbalanced partitions.

Justification

Quick Sort is faster than many sorting algorithms in practice due to low memory usage and excellent average-case performance. Randomized pivot selection reduces the probability of worst-case execution.

Q3. Case Study: Search Engine Query Processing (Binary Search)

Problem Statement

A search engine stores millions of keywords in sorted order and must retrieve search results quickly.

Analysis

Fast keyword lookup is essential to reduce user waiting time. Since the data is already sorted, Binary Search provides an efficient solution.

Key Requirements

- Sorted dataset.
- Fast retrieval.

- High efficiency for large datasets.
- Low memory usage.

Divide and Conquer Approach

Binary Search repeatedly divides the search space into two halves. It compares the target keyword with the middle element and continues searching only in the relevant half until the keyword is found.

Time Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(\log n)$
- **Worst Case:** $O(\log n)$
- **Space Complexity:** $O(1)$

Proposed Solution

Store keywords in sorted order and apply Binary Search whenever a user submits a search query.

Justification

Binary Search significantly reduces the number of comparisons, making it ideal for large search indexes and providing fast query processing.

Q4. Case Study: Image Processing System (Matrix Multiplication)

Problem Statement

An image processing application performs transformations using large matrices, requiring efficient matrix multiplication.

Analysis

Standard matrix multiplication becomes slow for high-resolution images because of the large number of arithmetic operations.

Key Challenges

- Large computational cost.
- High execution time.
- Efficient handling of large matrices.
- Reduced processing delay.

Divide and Conquer Approach

Strassen's Matrix Multiplication divides matrices into smaller submatrices and recursively multiplies them while reducing the total number of multiplication operations.

Time Complexity

- **Standard Matrix Multiplication:** $O(n^3)$
- **Strassen's Algorithm:** $O(n^{2.81})$

Proposed Solution

Use Strassen's Matrix Multiplication for large image matrices to reduce computation time.

Justification

Strassen's algorithm performs fewer multiplication operations than the standard method, making it more efficient for large matrices commonly used in image processing applications.

Q5. Case Study: Data Analytics Platform (Merge Sort vs Quick Sort)

Problem Statement

A data analytics platform processes massive datasets to generate reports quickly and accurately.

Analysis

The platform requires an efficient sorting algorithm while considering execution time, memory usage, and data stability.

Key Issues

- Large datasets.
- Memory consumption.
- Sorting speed.
- Stability of sorted data.

Comparison

Feature	Merge Sort	Quick Sort
Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n \log n)$	$O(n^2)$
Space Complexity	$O(n)$	$O(\log n)$
Stability	Stable	Not Stable

Divide and Conquer Approach

Both algorithms divide the dataset into smaller parts and recursively sort them. Merge Sort merges sorted subarrays, whereas Quick Sort partitions data around a pivot.

Performance Analysis

- Merge Sort provides consistent performance regardless of input.
- Quick Sort is generally faster for average cases due to better cache utilization.
- Merge Sort requires additional memory.

- Quick Sort uses less memory but may degrade to $O(n^2)$ with poor pivot selection.

Proposed Solution

- Use **Merge Sort** when stability and guaranteed performance are required.
- Use **Quick Sort** for in-memory sorting where speed and low memory usage are priorities.

Justification

For large-scale analytics involving stable and reliable sorting, **Merge Sort** is the preferred choice. For dynamic datasets where average-case speed is more important, **Quick Sort** offers better practical performance when implemented with randomized pivot selection.